BSc (Hons) in **Computer Science**

**Faculty of Computing**

# SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

## Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

## Part 1: Practical Implementation — Coding the Architectures

### Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates ( `agent_pos` ). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

### Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

### Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

## Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

**1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?**

For a table based agent, one row entry has to be provided for each unique percept sequence and not each state. In other words, there will be a separate row for every distinct history of percepts up until time t. As the number of entries increases as |P|^t, where P is the set of all percepts, the growth becomes exponential with increasing time t. The number of unique games in the classic estimation by Shannon for the game of Chess is about 10^120. This number exceeds the total number of atoms in our observable universe by about 10^40

**2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.**

In my 'SimpleReflexAgent', the 'sense_and_act' method contains three IF-THEN statements that directly represent Condition Action rules :

Rule 1 — IF food_here THEN Suck :
if percept['food_here']: return 'Suck' - If the agent is standing on a food pellet, it immediately sucks it up. The condition is 'food_here' and the action is 'Suck'.

Rule 2 — IF wall_ahead THEN turn_left :
if percept['wall_ahead']: return 'Left' - If there is a wall or boundary directly in front of the agent, it turns left. The condition is 'wall_ahead' and the action is 'Left'.

Rule 3 — ELSE move_forward :
return 'Forward' - If none of the above conditions are true, the agent moves forward by default.

Each rule checks only the current percept with no memory or history used. The agent simply matches the current sensor input to a fixed action, which is exactly how Condition Action rules work in a Simple Reflex Agent the percept is the condition and the returned value is the action.

**3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?**

The 'SimpleReflexAgent' got stuck in an infinite loop because of two reasons working together :

1. Partial Observability : The agent can only see 'wall_ahead' and 'food_here'. It cannot see its position, what is around it, or where it has already been. So the same percept can repeat in many different situations and the agent cannot tell the difference.

2. No Percept History : The agent has no '__init__', so it stores nothing between steps. Every decision is made purely from the current percept with no memory of past actions or failures.

How they cause the failure : When the agent hits a wall it turns left. If it ends up facing a wall again, it turns left again. Since it has no memory, it cannot detect it is repeating the same actions. It keeps reacting the same way to the same percept, causing it to loop forever
for example : turn left -> forward -> hit wall -> turn left -> forward -> hit wall. Without percept history, the agent is completely blind to the fact that it is stuck in a cycle.

**4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).**

In my 'ModelBasedAgent', the 'update_state' method handles both models and is always called first inside 'sense_and_act' before any decision is made.

Transition Model (how the world evolves based on actions) : After each action, the agent updates its internal state. If the last action was 'Forward' and no wall was hit, 'self.position' is updated to the new cell. If the last action was 'Left' or 'Right', 'self.facing' is updated. This tracks how the agent's actions change its position and direction in the world.

Sensor Model (how the agent interprets its percept) : The current percept 'wall_ahead' is used to verify whether a 'Forward' action actually succeeded. If 'wall_ahead' is True, the position is not updated, meaning the agent understands its move was blocked.

After updating the state, 'self.visited_cells' stores every cell visited. The decision rules then query this memory  if the cell ahead has already been visited, the agent turns instead of moving forward. This allows the agent to avoid loops and explore new paths, which the 'SimpleReflexAgent' could not do.

Finalize and Lock Lab 02 Documentation



FACULTY OF COMPUTING